

Building an Artificial Neural Network with PyTorch

Fashion-MNIST Data, Mini-Batch Training, and Evaluation

Abstract

An artificial neural network (ANN), also called a multilayer perceptron (MLP), can classify a flattened grayscale image by composing affine layers with nonlinear activation functions. These notes build a complete PyTorch workflow for a 6,000-image Fashion-MNIST subset: inspect and scale the data, split it into training and test partitions, implement the `Dataset` contract, construct mini-batch loaders, define a $784 \rightarrow 128 \rightarrow 64 \rightarrow 10$ network, optimize cross-entropy with stochastic gradient descent, and compute test accuracy from row-wise class predictions. Every important tensor shape, state transition, formula, and common failure mode is made explicit.

Contents

1	Purpose and complete mental map	2
2	Fashion-MNIST as a classification problem	3
2.1	Images, labels, and classes	3
2.2	Why a 785-column table appears	3
2.3	Why only 6,000 images are used initially	4
3	Dependencies, reproducibility, and data inspection	4
3.1	Imports and a fixed random seed	4
3.2	Load and inspect the table	5
3.3	Render the first sixteen images	5
4	Feature-target extraction, splitting, and scaling	5
5	The custom <code>Dataset</code> contract	6
6	Mini-batches with <code>DataLoader</code>	7
7	The 784-to-128-to-64-to-10 network	8
7.1	Layer purpose and shape flow	8
7.2	Expanded numerical example	8
7.3	PyTorch model definition	9
7.4	Parameter counting	10
8	Logits, softmax, and cross-entropy	10
8.1	From ten scores to probabilities	10
8.2	Why the model omits softmax	10

9	Configure learning	11
10	The mini-batch training state machine	12
10.1	Runnable loop	12
11	Evaluation state and accuracy	13
11.1	Two independent controls	13
11.2	Runnable evaluation loop	14
12	A compact end-to-end implementation	15
13	Common failures and diagnostic invariants	16
14	How the baseline can be improved	17
15	Key takeaways	18
16	Review questions	18

1 Purpose and complete mental map

The practical purpose is to turn an image containing one fashion item into one of ten class identifiers. The complete pipeline has three responsibilities:

1. **Data preparation:** represent each 28×28 image as 784 scaled features and preserve its integer class label.
2. **Learning:** repeatedly process mini-batches, measure prediction error, differentiate that error, and update trainable parameters.
3. **Evaluation:** freeze parameter updates, convert ten output scores per image into one class identifier, and count correct predictions.

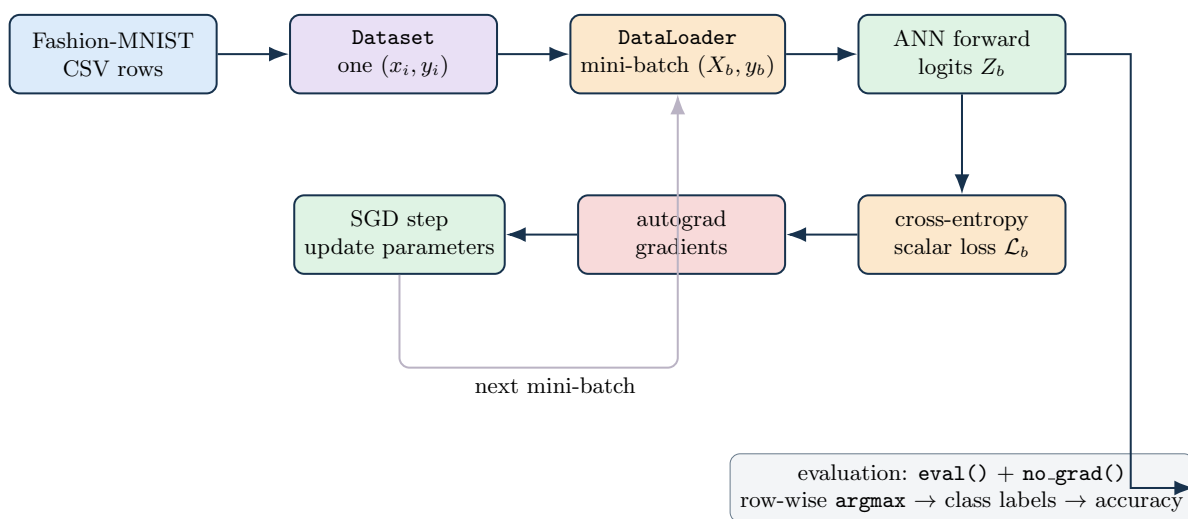


Figure 1: The complete workflow separates sample access, mini-batch iteration, differentiable learning, and evaluation. [Technical reference](#).

Six PyTorch ideas cooperate in this implementation:

Tensor A multidimensional numerical array with a data type, shape, and device.

Autograd PyTorch’s automatic differentiation system; it computes derivatives of a scalar loss with respect to trainable tensors.

nn.Module The base class that registers layers, parameters, and train/evaluation state.

Dataset The indexed sample-access contract: integer index i maps to one feature–label pair (x_i, y_i) .

DataLoader The iterable that samples indices, fetches samples, collates them into mini-batches, and optionally shuffles training order.

Optimizer An object that uses parameter gradients to mutate model parameters according to an update rule.

Takeaway

The model is only one component. A correct classifier also requires a shape-safe data pipeline, a correctly ordered training state machine, and a separate evaluation procedure.

2 Fashion-MNIST as a classification problem

2.1 Images, labels, and classes

Fashion-MNIST contains 70,000 grayscale images in total: 60,000 in its standard training split and 10,000 in its standard test split. Every image has height 28 pixels and width 28 pixels. A *pixel* is one image location whose grayscale intensity is represented by an integer from 0 to 255.

The target is one integer class identifier:

ID	Class	ID	Class
0	T-shirt/top	5	Sandal
1	Trouser	6	Shirt
2	Pullover	7	Sneaker
3	Dress	8	Bag
4	Coat	9	Ankle boot

This is a **multiclass classification** problem because exactly one class must be selected from ten mutually exclusive possibilities.

2.2 Why a 785-column table appears

A dense ANN expects a feature vector rather than a two-dimensional grid. Flattening places the 784 pixel values into one ordered vector:

$$x = \begin{bmatrix} x_1 & x_2 & \cdots & x_{784} \end{bmatrix} \in \mathbb{R}^{784}.$$

Here x_j is the intensity of flattened pixel position j . The symbol $\mathbb{R}^{784}$ means a vector with 784 real-valued entries. One CSV row therefore has 785 columns: one label plus 784 pixel columns.

Flattening preserves every intensity but removes the explicit two-dimensional neighbourhood relationship between nearby pixels. A fully connected network can still learn useful patterns, but a convolutional neural network can exploit spatial structure more directly.

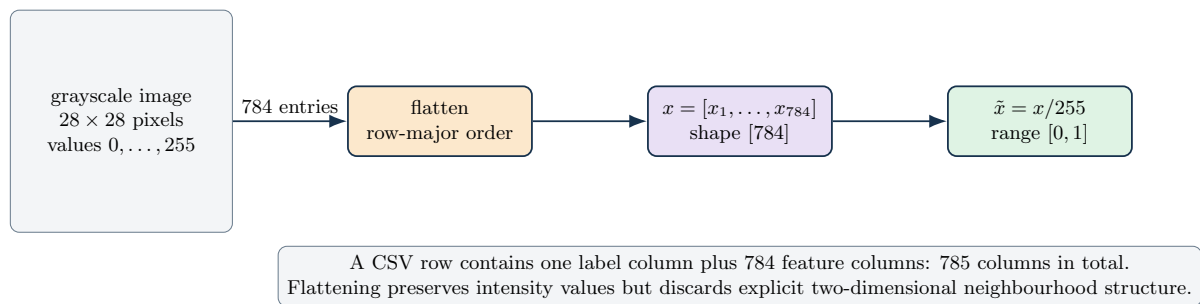


Figure 2: A 28×28 grayscale image becomes a 784-feature vector and is scaled to the interval $[0, 1]$. [Technical reference](#).

2.3 Why only 6,000 images are used initially

A 6,000-image subset makes a complete CPU training cycle faster while preserving the full data flow. An 80:20 split produces

$$N_{\text{train}} = 0.8 \times 6000 = 4800, \quad N_{\text{test}} = 0.2 \times 6000 = 1200.$$

The training partition teaches the parameters. The test partition estimates performance on held-out examples and must not influence parameter updates.

Pitfall

Repeatedly changing hyperparameters after inspecting test accuracy turns the test set into a tuning set. A robust project uses separate training, validation, and test partitions; the test partition is reserved for the final estimate.

3 Dependencies, reproducibility, and data inspection

3.1 Imports and a fixed random seed

```
import pandas as pd
from sklearn.model_selection import train_test_split

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader

import matplotlib.pyplot as plt

# Make PyTorch initialization and PyTorch random operations repeatable.
torch.manual_seed(42)
```

Listing 1: Import the required libraries and seed PyTorch

The integer 42 has no mathematical advantage; it merely fixes a pseudorandom sequence. Complete reproducibility can additionally require seeding NumPy and Python, controlling DataLoader generators and workers, selecting deterministic algorithms, and recording software/hardware versions.

3.2 Load and inspect the table

```
df = pd.read_csv("fmnist_small.csv")
print(df.shape)           # expected for this subset: (6000, 785)
print(df.head())          # label, pixel1, ..., pixel784
```

Listing 2: Load the Fashion-MNIST subset and inspect its columns

The table preview verifies three properties before training: the target column is named `label`, the remaining columns are pixels, and the row/column counts match the intended subset.

3.3 Render the first sixteen images

```
fig, axes = plt.subplots(4, 4, figsize=(10, 10))
fig.suptitle("First 16 Images", fontsize=16)

for i, ax in enumerate(axes.flat):
    image = df.iloc[i, 1:].values.reshape(28, 28)
    label = df.iloc[i, 0]
    ax.imshow(image, cmap="gray")
    ax.axis("off")
    ax.set_title(f"Label: {label}")

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
```

Listing 3: Reshape and display the first sixteen flattened images

The slice `df.iloc[i, 1:]` excludes the label and keeps 784 values. Reshaping to $(28, 28)$ is valid because $28 \cdot 28 = 784$. Inspection can expose swapped columns, implausible labels, corrupted intensities, or an incorrect reshape order before those defects enter the model.

Shape check

For one row, `df.iloc[i, 1:].values` has shape $(784,)$. After `reshape(28, 28)`, the number of entries remains 784; only the interpretation of the axes changes.

4 Feature–target extraction, splitting, and scaling

```
X = df.iloc[:, 1:].values
Y = df.iloc[:, 0].values

X_train, X_test, y_train, y_test = train_test_split(
    X,
    Y,
    test_size=0.2,
    random_state=42,
)

X_train = X_train / 255.0
X_test = X_test / 255.0
```

Listing 4: Separate labels and pixels, split the rows, and scale intensities

Capital X conventionally denotes a feature matrix and lowercase y a target vector. The names Y and y are case-sensitive in Python; one naming convention should be used consistently.

The shape contract is

$$X_{\text{train}} \in \mathbb{R}^{4800 \times 784}, \quad y_{\text{train}} \in \{0, \dots, 9\}^{4800},$$

$$X_{\text{test}} \in \mathbb{R}^{1200 \times 784}, \quad y_{\text{test}} \in \{0, \dots, 9\}^{1200}.$$

The first matrix axis counts examples; the second counts features. Each target vector has one class identifier per matrix row.

Scaling uses the known pixel range:

$$\tilde{x}_j = \frac{x_j}{255}.$$

Here $x_j \in \{0, \dots, 255\}$ is one original intensity and $\tilde{x}_j \in [0, 1]$ is its scaled value. For example,

$$0/255 = 0, \quad 128/255 \approx 0.502, \quad 255/255 = 1.$$

The transformation does not change shape or class information. It reduces feature magnitude, which usually makes optimization better conditioned than operating directly on values as large as 255.

Intuition

Gradient-based optimization is easier to control when input features have comparable, moderate scales. Dividing by 255 is appropriate here because grayscale intensity has a known fixed range; it does not need statistics estimated from the test set.

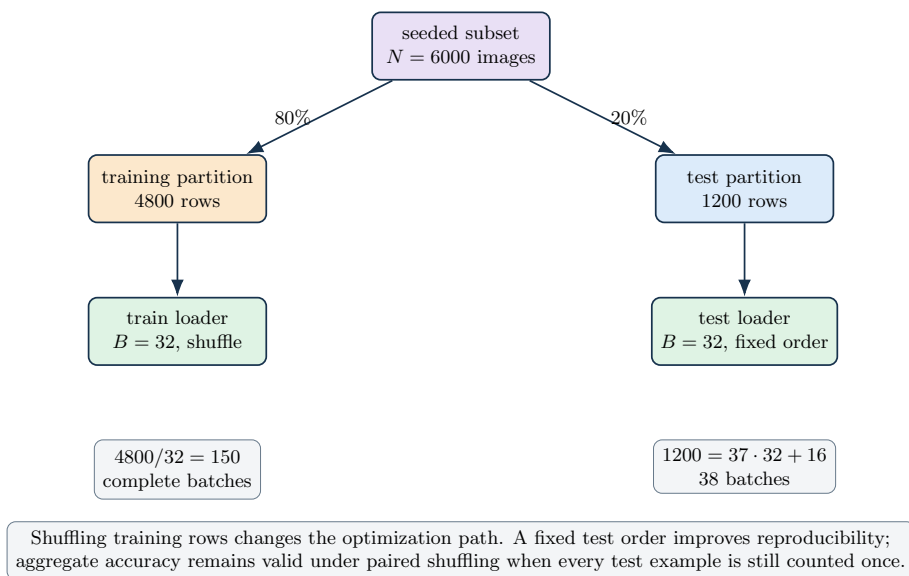


Figure 3: A seeded 80:20 split produces 150 complete training batches and 38 test batches. [Technical reference](#).

An optional improvement is `stratify=Y`, which keeps approximately the same class proportions in both partitions. It is useful when a small random subset might otherwise contain noticeably unequal class counts.

5 The custom Dataset contract

A map-style `Dataset` defines how an integer index retrieves one aligned sample. Three methods establish the contract:

- `__init__` stores and converts the complete arrays;
- `__len__` reports the sample count;
- `__getitem__` returns the feature and label at one index.

```

class CustomDataset(Dataset):
    def __init__(self, features, labels):
        self.features = torch.tensor(features, dtype=torch.float32)
        self.labels = torch.tensor(labels, dtype=torch.long)

    def __len__(self):
        return len(self.features)

    def __getitem__(self, index):
        return self.features[index], self.labels[index]

```

Listing 5: Convert NumPy arrays into a map-style PyTorch Dataset

Feature data type. Ordinary `nn.Linear` parameters use floating-point values. Features are therefore converted to `float32`. A byte tensor of raw pixel values cannot be multiplied directly by floating-point layer weights.

Target data type. `nn.CrossEntropyLoss` expects class indices in integer `long` format. Each label is a scalar whose value lies from 0 through 9.

Aligned indexing. Both stored tensors are indexed with the same `index`. Shuffling must reorder sample indices, not independently permute features and labels.

```

train_dataset = CustomDataset(X_train, y_train)
test_dataset = CustomDataset(X_test, y_test)

feature_0, label_0 = train_dataset[0]
print(len(train_dataset))      # 4800
print(len(test_dataset))      # 1200
print(feature_0.shape)        # torch.Size([784])
print(label_0.shape)          # torch.Size([]): scalar tensor
print(feature_0.dtype)        # torch.float32
print(label_0.dtype)          # torch.int64

```

Listing 6: Construct training and test datasets and inspect one sample

Pitfall

Python's special method names require two leading and two trailing underscores. Misspelling `__len__` or `__getitem__` prevents Python or `DataLoader` from invoking the intended method.

6 Mini-batches with DataLoader

```

train_loader = DataLoader(
    train_dataset,
    batch_size=32,
    shuffle=True,
)

test_loader = DataLoader(
    test_dataset,
    batch_size=32,
    shuffle=False,
)

```

Listing 7: Create shuffled training batches and deterministic test batches

With batch size $B = 32$, an ordinary training batch has

$$X_b \in \mathbb{R}^{32 \times 784}, \quad y_b \in \{0, \dots, 9\}^{32}.$$

The subscript b identifies one mini-batch. Since $4800/32 = 150$, every training batch is complete. The test partition has 37 complete batches and one final batch of 16, giving 38 batches.

Training shuffle. Changing row order between epochs prevents a fixed local ordering from controlling every mini-batch and changes the stochastic optimization path.

Test order. A fixed order improves reproducibility and makes per-example outputs easier to trace. Shuffling paired test samples would not mathematically invalidate aggregate accuracy if every example were still visited exactly once; the important invariant is feature-label alignment and complete counting.

Shape check

Before defining the network, inspect one batch: `batch_features.shape == (32, 784)` and `batch_labels.shape == (32,)`. A mismatch here should be repaired before training.

7 The 784-to-128-to-64-to-10 network

7.1 Layer purpose and shape flow

A fully connected or *dense* layer gives every output unit a learned connection to every input feature. For a batch $X \in \mathbb{R}^{B \times d_{\text{in}}}$, `nn.Linear` computes

$$Z = XW^T + b.$$

Here

- B is batch size;
- d_{in} is the input-feature count;
- d_{out} is the output-unit count;
- $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is the weight matrix;
- $b \in \mathbb{R}^{d_{\text{out}}}$ is a bias vector broadcast across all B rows;
- $Z \in \mathbb{R}^{B \times d_{\text{out}}}$ is the layer output.

A ReLU activation then computes

$$\text{ReLU}(z) = \max(0, z)$$

elementwise. ReLU preserves shape but replaces each negative value with zero. Nonlinearity matters: several affine maps with no activation between them collapse algebraically into one affine map.

7.2 Expanded numerical example

Let one input be $x = [2, -1]$, with

$$W = \begin{bmatrix} 1 & 2 \\ -1 & 1 \end{bmatrix}, \quad b = \begin{bmatrix} 0 & 1 \end{bmatrix}.$$

The first output is $2(1) + (-1)(2) + 0 = 0$. The second is $2(-1) + (-1)(1) + 1 = -2$. Therefore

$$xW^T + b = \begin{bmatrix} 0 & -2 \end{bmatrix}, \quad \text{ReLU}(xW^T + b) = \begin{bmatrix} 0 & 0 \end{bmatrix}.$$

The same operations are vectorized across every row of a mini-batch.

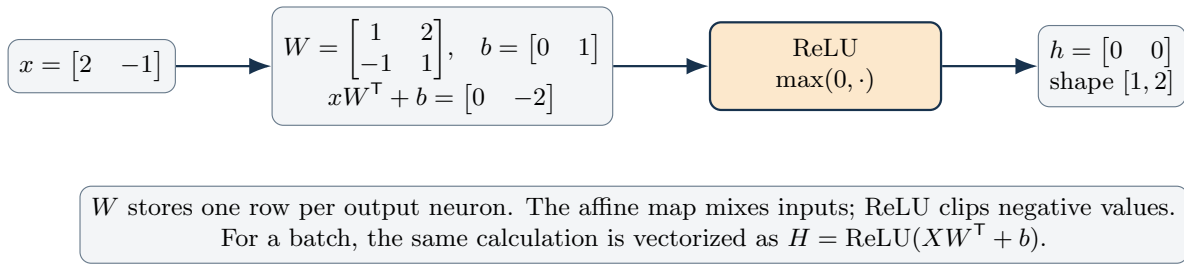


Figure 4: A small affine calculation illustrates weight orientation, bias addition, and elementwise ReLU. [Technical reference](#).

7.3 PyTorch model definition

```
class MyNN(nn.Module):
    def __init__(self, num_features):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(num_features, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 10),
        )

    def forward(self, features):
        return self.model(features)
```

Listing 8: Define the fully connected classifier with a sequential container

`super().__init__()` initializes the parent `nn.Module` registries. Assigning the sequential container to `self.model` registers every nested weight and bias. Calling `model(batch_features)` dispatches to `forward` through PyTorch’s module machinery.

For batch size B , the complete shape flow is

$$[B, 784] \longrightarrow [B, 128] \longrightarrow [B, 128] \longrightarrow [B, 64] \longrightarrow [B, 64] \longrightarrow [B, 10].$$

The two repeated shapes occur because ReLU changes values but not dimensions. The final ten values are **logits**: unconstrained real-valued class scores.

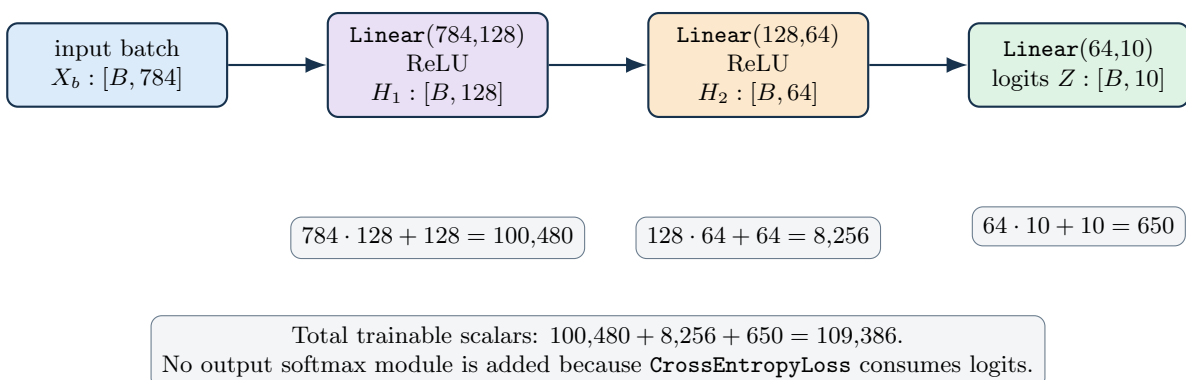


Figure 5: The classifier transforms 784 input features through two ReLU hidden layers into ten logits. [Technical reference](#).

7.4 Parameter counting

An affine layer with d_{in} inputs and d_{out} outputs has

$$d_{\text{out}}d_{\text{in}} + d_{\text{out}}$$

trainable scalars: one weight for every input–output connection and one bias per output. Thus,

$$784 \rightarrow 128 : 784 \cdot 128 + 128 = 100,480,$$

$$128 \rightarrow 64 : 128 \cdot 64 + 64 = 8,256,$$

$$64 \rightarrow 10 : 64 \cdot 10 + 10 = 650.$$

The total is

$$100,480 + 8,256 + 650 = 109,386.$$

ReLU contributes no trainable parameters.

8 Logits, softmax, and cross-entropy

8.1 From ten scores to probabilities

For one example, let $z = [z_0, \dots, z_9]$ be ten logits. Softmax converts them into probabilities:

$$p_k = \frac{e^{z_k}}{\sum_{j=0}^9 e^{z_j}}, \quad k \in \{0, \dots, 9\}.$$

The index k denotes one class, $p_k \in (0, 1)$ is its probability, and $\sum_k p_k = 1$.

For the smaller three-class example $z = [1.2, -0.2, 2.1]$,

$$p \approx [0.270, 0.067, 0.664].$$

If the true class is $y = 2$, cross-entropy is

$$\ell(z, y) = -\log p_y = -\log(0.664) \approx 0.410.$$

The loss is small when the true class receives high probability and large when it receives low probability.

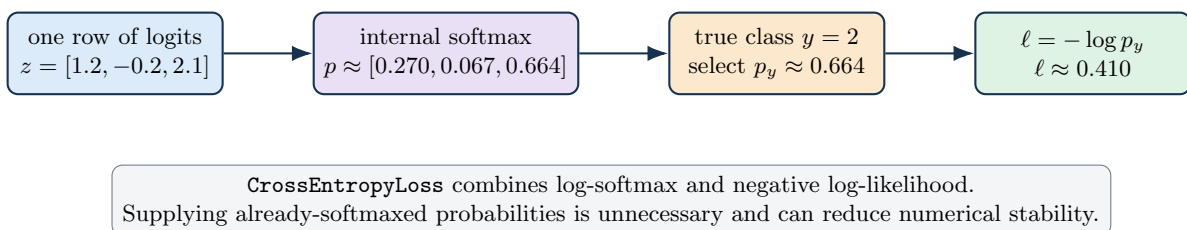


Figure 6: Cross-entropy transforms logits internally, selects the true class, and returns a scalar penalty. [Technical reference](#).

8.2 Why the model omits softmax

`nn.CrossEntropyLoss` expects raw logits and internally combines log-softmax with negative log-likelihood. Adding `nn.Softmax` to the model output before this loss is unnecessary and can reduce numerical stability.

For prediction, softmax is also unnecessary because it preserves ordering:

$$\arg \max_k z_k = \arg \max_k p_k.$$

The class with the largest logit also has the largest softmax probability.

Pitfall

A tensor of shape $[B, 10]$ returned by this model contains logits, not probabilities. Its rows need not sum to one, and entries may be negative. Call `torch.softmax(logits, dim=1)` only when actual probabilities are required for interpretation.

9 Configure learning

```
epochs = 100
learning_rate = 0.1

model = MyNN(X_train.shape[1])
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(
    model.parameters(),
    lr=learning_rate,
)
```

Listing 9: Instantiate the model, loss function, and SGD optimizer

An **epoch** is one complete pass through all 150 training mini-batches. The **learning rate** $\eta = 0.1$ controls update magnitude. `model.parameters()` supplies all 109,386 registered trainable scalars to the optimizer.

For plain stochastic gradient descent (SGD), each parameter tensor θ is updated by

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}_t.$$

Here t is the mini-batch update index, $\mathcal{L}_t$ is the current batch loss, and $\nabla_{\theta} \mathcal{L}_t$ is the loss derivative with respect to θ . The derivative indicates the direction of local loss increase; subtracting it aims to reduce loss.

With 150 mini-batches per epoch and 100 epochs, the optimizer performs

$$150 \times 100 = 15,000$$

parameter-update steps.

10 The mini-batch training state machine

10.1 Runnable loop

```

model.train()

for epoch in range(epochs):
    total_epoch_loss = 0.0

    for batch_features, batch_labels in train_loader:
        optimizer.zero_grad()
        outputs = model(batch_features)
        loss = criterion(outputs, batch_labels)
        loss.backward()
        optimizer.step()

        total_epoch_loss += loss.item()

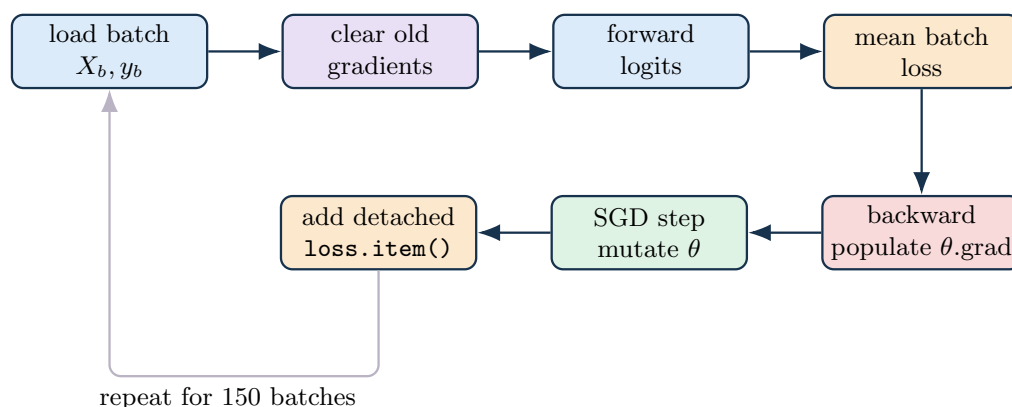
    average_loss = total_epoch_loss / len(train_loader)
    print(f"Epoch: {epoch + 1}, Loss: {average_loss:.4f}")

```

Listing 10: Train for 100 epochs and report mean mini-batch loss

The order has a precise state meaning:

1. `optimizer.zero_grad()` clears derivatives accumulated by the preceding backward pass.
2. `model(batch_features)` computes logits of shape `[32, 10]` for a complete batch.
3. `criterion(outputs, batch_labels)` computes one scalar mean cross-entropy loss. Labels have shape `[32]` and dtype `long`.
4. `loss.backward()` traverses the autograd graph and populates each parameter's `.grad` tensor.
5. `optimizer.step()` mutates every managed parameter according to SGD.
6. `loss.item()` extracts a detached Python number for reporting; it is not used to differentiate.



After one epoch, report the average of 150 batch losses. Repeat for 100 epochs. Gradient buffers must be cleared exactly once before each ordinary backward pass.

Figure 7: One training iteration clears gradient state, computes a loss, differentiates it, and mutates parameters. [Technical reference](#).

The source ordering may clear gradients immediately before `backward()` rather than before the forward

pass. Both are correct if gradient buffers are cleared exactly once after the preceding update and before the next backward pass. Clearing at the start of the mini-batch is easier to audit.

The reported quantity

$$\bar{\mathcal{L}}_{\text{epoch}} = \frac{1}{150} \sum_{b=1}^{150} \mathcal{L}_b$$

is the mean of 150 batch losses. Here all training batches contain 32 examples, so it also equals the per-example mean for the epoch. If batch sizes differed, an exact epoch mean would weight each batch loss by its sample count.

A strong downward loss trend is evidence that optimization is fitting the training objective. One seeded run reported 1.3216 at epoch 1 and 0.00831 at epoch 100. Individual epochs still increased—for example, late losses moved from about 0.0174 to 0.0678 before falling again—because shuffled mini-batch optimization is noisy. Training loss does not by itself prove good generalization; test or validation performance supplies that evidence.

Pitfall

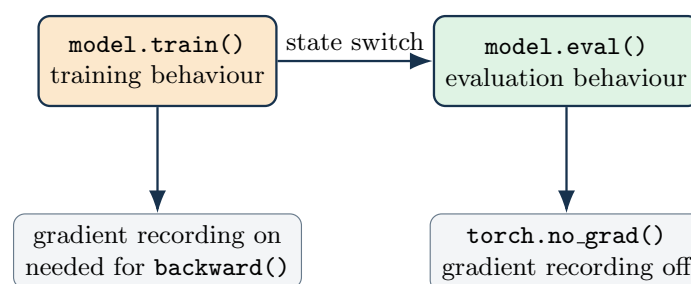
PyTorch accumulates gradients by default. Omitting `zero_grad()` adds new derivatives to old buffers and changes the effective update. Gradient accumulation is useful only when deliberately designed across several mini-batches.

11 Evaluation state and accuracy

11.1 Two independent controls

`model.eval()` switches stateful modules to evaluation behaviour. Dropout stops randomly suppressing activations, and batch normalization uses stored running statistics instead of current-batch statistics. This particular architecture contains neither component, but setting the state establishes a correct reusable pattern.

`torch.no_grad()` independently disables autograd graph construction. It reduces memory and computational work because evaluation does not call `backward()`.



Evaluation state changes dropout and batch-normalization behaviour; it does not disable autograd. The two controls solve different problems and should be used together during inference.

Figure 8: Evaluation state and disabled gradient recording solve different problems and should be used together. [Technical reference](#).

11.2 Runnable evaluation loop

```

model.eval()
total = 0
correct = 0

with torch.no_grad():
    for batch_features, batch_labels in test_loader:
        outputs = model(batch_features)          # logits: [B, 10]
        _, predicted = torch.max(outputs, dim=1)

        total += batch_labels.shape[0]
        correct += (predicted == batch_labels).sum().item()

accuracy = correct / total
print(f"Accuracy: {100 * accuracy:.2f}%")

```

Listing 11: Compute exact sample-level test accuracy across all mini-batches

`torch.max(outputs, dim=1)` operates across the ten class columns. It returns two tensors: maximum values and their column indices. The underscore discards the values; `predicted` keeps the class indices and has shape $[B]$.

For example, consider a three-example, four-class logit matrix:

$$Z = \begin{bmatrix} 2.5 & 1.0 & 0.5 & 3.0 \\ 0.1 & 2.3 & 0.8 & 0.5 \\ 1.2 & 1.5 & 2.8 & 0.3 \end{bmatrix}.$$

Row-wise argmax produces

$$\hat{y} = \begin{bmatrix} 3 & 1 & 2 \end{bmatrix}.$$

If the true labels are $y = [3, 0, 2]$, the first and third predictions are correct, so accuracy is

$$\text{accuracy} = \frac{\sum_{i=1}^N \mathbf{1}[\hat{y}_i = y_i]}{N} = \frac{2}{3} \approx 66.7\%.$$

Here N is the number of evaluated examples and $\mathbf{1}[\cdot]$ equals 1 when its condition is true and 0 otherwise.

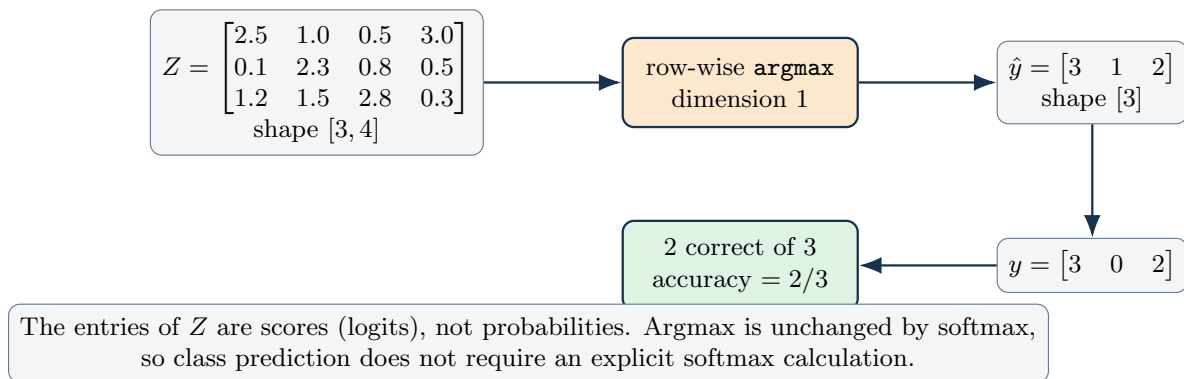


Figure 9: Row-wise argmax converts logits to labels, which are compared elementwise with the targets. [Technical reference.](#)

The 1,200-example test set is delivered as 37 batches of 32 and one batch of 16. Incrementing `total` by the actual batch size ensures the final smaller batch is counted correctly. Accumulating correct predictions and total examples is preferable to averaging 38 unweighted batch accuracies, which would over-weight the final 16-example batch.

A seeded run of this configuration printed 0.8325, or 83.25% test accuracy. The exact value is not a universal expected output: subset selection, initialization, library versions, and numerical execution can change the optimization path.

12 A compact end-to-end implementation

The following two blocks gather the data and model responsibilities at named boundaries without hiding shape conversions.

```
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.model_selection import train_test_split
from torch.utils.data import Dataset, DataLoader

torch.manual_seed(42)
df = pd.read_csv("fmnist_small.csv")
X = df.iloc[:, 1:].values
y = df.iloc[:, 0].values
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
X_train, X_test = X_train / 255.0, X_test / 255.0

class CustomDataset(Dataset):
    def __init__(self, features, labels):
        self.features = torch.tensor(features, dtype=torch.float32)
        self.labels = torch.tensor(labels, dtype=torch.long)
    def __len__(self):
        return len(self.features)
    def __getitem__(self, index):
        return self.features[index], self.labels[index]

train_data = CustomDataset(X_train, y_train)
test_data = CustomDataset(X_test, y_test)
train_loader = DataLoader(train_data, batch_size=32, shuffle=True)
test_loader = DataLoader(test_data, batch_size=32, shuffle=False)
```

Listing 12: Part 1: prepare datasets, loaders, and the model

```

class MyNN(nn.Module):
    def __init__(self, num_features):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(num_features, 128), nn.ReLU(),
            nn.Linear(128, 64), nn.ReLU(),
            nn.Linear(64, 10),
        )
    def forward(self, features):
        return self.model(features)

model = MyNN(X_train.shape[1])
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.1)

for epoch in range(100):
    model.train()
    epoch_loss = 0.0
    for features, labels in train_loader:
        optimizer.zero_grad()
        logits = model(features)
        loss = criterion(logits, labels)
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item()
    print(epoch + 1, epoch_loss / len(train_loader))

model.eval()
correct = total = 0
with torch.no_grad():
    for features, labels in test_loader:
        predicted = model(features).argmax(dim=1)
        correct += (predicted == labels).sum().item()
        total += labels.numel()
print(f"Accuracy: {100 * correct / total:.2f}%")

```

Listing 13: Part 2: define, train, and evaluate the classifier

13 Common failures and diagnostic invariants

1. **Wrong feature dtype.** A uint8 input and float32 weights cause a matrix-multiplication dtype error. Convert features to float32.
2. **Wrong target dtype.** Floating targets cause `CrossEntropyLoss` to reject class-index labels. Convert them to long.
3. **Wrong target shape.** For ordinary multiclass classification, logits are $[B, 10]$ and class-index targets are $[B]$, not $[B, 1]$.
4. **Out-of-range labels.** Every target must lie from 0 through 9. A value of 10 is invalid for ten output classes.
5. **Wrong flatten size.** A layer expecting 784 inputs fails if images remain $[B, 28, 28]$. Flatten to $[B, 784]$, for example with `features.view(features.shape[0], -1)` when needed.
6. **Explicit softmax before cross-entropy.** Supply logits directly to `CrossEntropyLoss`.
7. **Missing gradient reset.** Clear ordinary gradient buffers before every backward pass.
8. **Calling forward directly.** Prefer `model(features)` so PyTorch's module call machinery and hooks are preserved.

9. **Incomplete evaluation setup.** First call `model.eval()` to select module behaviour. Then disable gradient recording with `no_grad()`.
10. **Ignoring a smaller final batch.** Derive counters and reshapes from the actual batch size rather than assuming every test batch has 32 rows.
11. **Feature-label misalignment.** Shuffle sample indices through `DataLoader`; never shuffle feature and target arrays independently.
12. **Treating training loss as generalization.** Track held-out metrics and avoid tuning repeatedly on the final test set.

A useful pre-training invariant block is:

```
features, labels = next(iter(train_loader))
logits = model(features)

assert features.shape[1] == 784
assert features.dtype == torch.float32
assert labels.ndim == 1
assert labels.dtype == torch.long
assert labels.min().item() >= 0
assert labels.max().item() < 10
assert logits.shape == (features.shape[0], 10)
assert sum(p.numel() for p in model.parameters()) == 109_386
```

Listing 14: Assert the critical shape, dtype, and range contracts

14 How the baseline can be improved

The baseline uses a small subset, CPU execution, plain SGD, one learning rate, 100 epochs, default initialization, and a fixed architecture. Improvement levers include:

1. **More data:** train on the full collection rather than only 6,000 examples.
2. **Faster compute:** move the model and every batch tensor to a GPU so larger experiments finish sooner.
3. **Optimizer choice:** compare SGD, momentum, Adam, and RMSprop under a controlled validation protocol.
4. **Learning-rate design:** tune the initial rate or use a scheduler rather than assuming 0.1 is optimal.
5. **Training duration:** monitor validation behaviour; more epochs can help or cause overfitting.
6. **Initialization:** choose a scheme appropriate for ReLU layers, such as Kaiming initialization.
7. **Regularization:** evaluate weight decay and dropout when training and validation performance diverge.
8. **Normalization:** batch normalization can change optimization dynamics, but its train/evaluation state must be managed correctly.
9. **Architecture:** tune width and depth, or use a convolutional network that preserves image spatial structure.

A fair comparison changes one factor at a time or uses a reproducible hyperparameter search, retains a fixed validation protocol, records seeds and versions, and reports more than one random run when variability matters.

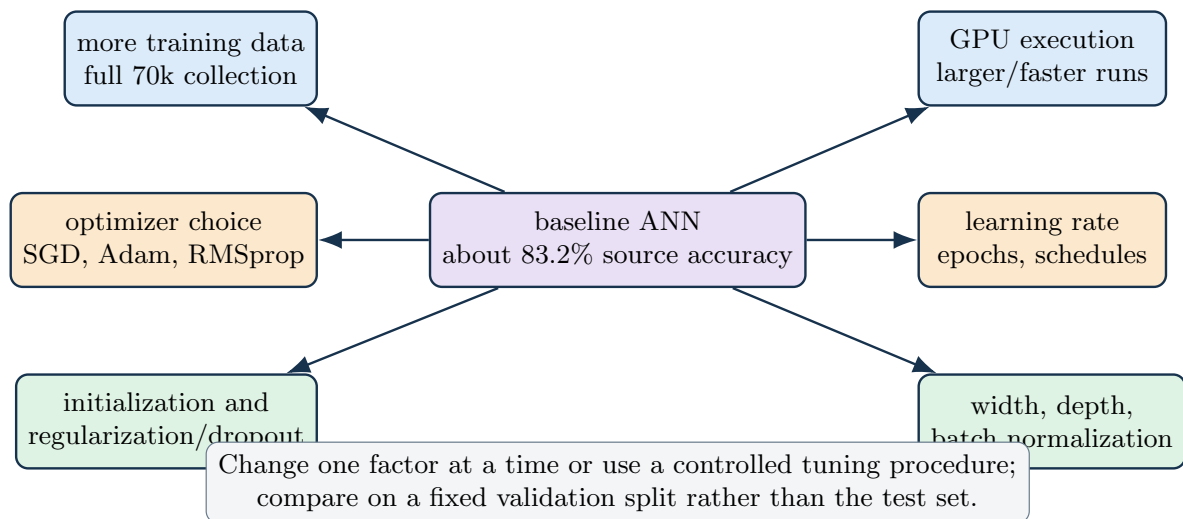


Figure 10: Data, compute, optimization, regularization, and architecture provide distinct improvement levers. [Technical reference](#).

15 Key takeaways

1. A Fashion-MNIST CSV row contains one class label and 784 flattened grayscale intensities.
2. Dividing intensities by 255 maps their known range to $[0, 1]$ without changing tensor shape.
3. A custom Dataset converts features to `float32`, labels to `long`, reports length, and returns aligned indexed samples.
4. With 4,800 training rows and batch size 32, one epoch contains 150 mini-batches; 1,200 test rows produce 38 batches because the final batch has 16 rows.
5. The $784 \rightarrow 128 \rightarrow 64 \rightarrow 10$ MLP has 109,386 trainable parameters.
6. ReLU introduces nonlinearity; the final layer returns ten logits rather than probabilities.
7. `CrossEntropyLoss` consumes logits and integer class indices and internally performs the stable softmax-related calculation.
8. Every ordinary training iteration must clear gradients, run a forward pass, compute scalar loss, backpropagate, and update parameters.
9. `model.eval()` and `torch.no_grad()` are complementary rather than interchangeable.
10. Accuracy should aggregate correct and total sample counts so a smaller final batch receives the correct weight.

16 Review questions

1. Why does a 28×28 image produce 784 input features, and what information does flattening make less explicit?
2. What shapes and dtypes should one sample and one 32-example batch have?
3. Why must multiclass class-index targets use `torch.long`?
4. How are 109,386 parameters obtained for the three affine layers?
5. Why would removing both ReLU operations make the stacked affine layers no more expressive than one affine layer?

6. What is the difference between logits and softmax probabilities?
7. Why should softmax not be inserted before `CrossEntropyLoss`?
8. What state does `loss.backward()` create, and what state does `optimizer.step()` mutate?
9. Why are `model.eval()` and `torch.no_grad()` both needed in a general evaluation pipeline?
10. Why is sample-level aggregation safer than an unweighted mean of test-batch accuracies?
11. Which improvement choices require a validation set rather than repeated comparison on the final test set?